1. Create Generic Classes Using a Type Parameter (Interview Notes)

What is a Generic Class?

A **generic class** is a class that uses a **type parameter** (such as T) instead of a specific data type.

It allows the same class to work with different data types while maintaining **compile-time type safety**.

Example

```
class Box {

    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

Using the class:

```
Box box1 = new Box<>();
box1.set("Java");

Box box2 = new Box<>();
box2.set(100);

System.out.println(box1.get());
System.out.println(box2.get());
```

Output

```
Java
100
```

The same class works with different data types without duplication.

Why Use Generic Classes?

Without generics:

```
class Box {
    Object value;
}
```

Every retrieval requires casting.

With generics:

```
Box box = new Box<>();
```

No casting is needed, and the compiler ensures type safety.

Important Interview Points ⭐

- A generic class is declared using a **type parameter** like .
- T is a placeholder for the actual type supplied during object creation.
- Generics provide **compile-time type safety** and eliminate unnecessary casting.
- A single generic class can work with **multiple data types**, improving code reusability.
- Type parameter names are conventions:
    - T → Type
    - E → Element
    - K → Key
    - V → Value
    - N → Number
- Multiple type parameters are supported.

Example:

```
class Pair {
}
```

Tricky Interview Questions

Q1. What does mean?

Answer:

T is a **type parameter** that represents a type specified when the class is instantiated.

Q2. Can a generic class have multiple type parameters?

Answer: Yes.

```
class Pair {
}
```

Q3. Is T a keyword?

Answer: No.

It is simply a naming convention. You could write:

```
class Box {
}
```

Q4. Why are generic classes preferred over using Object?

Answer:

They provide compile-time type safety and eliminate explicit casting.

Q5. Can we create an object of a type parameter?

```
T obj = new T();
```

Answer: No.

Java uses **type erasure**, so the actual type is unavailable at runtime.

Interview Summary ⭐

- **Generic classes allow one class to work with multiple data types using type parameters.**
- They improve **code reusability**, **type safety**, and **readability**.
- They eliminate explicit casting by performing type checking at compile time.
- Generic classes are heavily used throughout the Java Collections Framework.

> **Interview One-Liner:**
>
> **"A generic class uses type parameters to create reusable, type-safe code that works with different data types without explicit casting."**

---

2. Create Generic Methods for Reusable Logic (Interview Notes)

What is a Generic Method?

A **generic method** declares its own type parameter and can work with **different data types**, regardless of whether the class itself is generic.

Example

```
class Utility {

    public static  void print(T value) {
```

```
        System.out.println(value);
    }
}
```

Using the method:

```
Utility.print("Java");
Utility.print(100);
Utility.print(3.14);
```

Output

```
Java
100
3.14
```

The same method works for multiple data types.

Why Use Generic Methods?

Instead of writing separate methods:

```
printString(String s);

printInteger(Integer i);

printDouble(Double d);
```

A single generic method handles all types.

Syntax

```
public static  void methodName(T value)
```

Here:

- declares the type parameter.
- T value uses that type.

Important Interview Points ⭐

- A generic method declares its own type parameter **before the return type**.
- Generic methods can exist inside **generic or non-generic classes**.
- The compiler usually **infers the type automatically**, so explicit type arguments are rarely needed.
- Generic methods improve **code reuse** and **type safety**.
- They eliminate the need for overloaded methods that differ only by parameter type.

Tricky Interview Questions

Q1. Where is the type parameter declared?

Answer:

Before the return type.

```
public  void display(T value)
```

Q2. Can a non-generic class contain a generic method?

Answer: Yes.

A generic method is independent of whether the class itself is generic.

Q3. Can a generic method return a generic type?

Answer: Yes.

```
public static  T getValue(T value) {
    return value;
}
```

Q4. Can generic methods have multiple type parameters?

Answer: Yes.

```
public static  void print(K key, V value) {
}
```

Q5. Does Java always require specifying the type explicitly?

```
Utility.print("Java");
```

Answer: No.

The compiler usually infers the type automatically:

```
Utility.print("Java");
```

Interview Summary ⭐

- **Generic methods use type parameters to make methods reusable across different data types.**
- They can be declared inside **generic or non-generic classes**.
- The type parameter is declared **before the return type**.
- Java usually infers the type automatically, making generic methods easy to use.
- Generic methods reduce duplication while maintaining compile-time type safety.

> **Interview One-Liner:**
>
> **"A generic method declares its own type parameter, allowing one method to work safely with multiple data types while reducing code duplication."**